

Assignment 1

Group Details:

Group No: 173

Sl. No.	BITS ID	Name	Contribution of Team Member (Qualitative)	Contribution of Team Member (Quantitative)
1.	2025AA05957	ABHISHEK	Code Review and GR4ML documentation	25
2.	2025AA05729	ASHMIT BHANDARI	FronEnd and BackEnd	25
3.	2025AA05478	RISHABH JAIN	Dataset Selection and Model Training	25
4.	2025AB05319	UDIT SHARMA	System Architecture	25

Important Links

Services	Link
GitHub Repo	https://github.com/Rjain3108/Health-Insurance-Cross-Sell-Prediction
Streamlit App	https://health-insurance-cross-sell-prediction.streamlit.app/
Fast API	https://health-insurance-cross-sell-prediction.onrender.com/docs

Project Details

Objective 1: Requirements Formulation

1. Select a domain and define an appropriate problem statement for an ML-based application of your choice.

1.1 Domain

Insurance

1.2 Project Title

Health Insurance Cross-Sell Prediction System

1.3 Problem Statement

An insurance company currently offers health insurance to its customers. The company wants to identify customers who are most likely to purchase vehicle insurance. Instead of contacting every customer, the marketing team wants to prioritize customers with the highest purchase probability using Machine Learning.

Business Problem

Current Challenges:

- Marketing campaigns target all customers.
- Low conversion rates.
- High customer acquisition cost.
- Inefficient resource utilization.

ML Solution

Develop an AI-powered recommendation system that predicts whether a customer will purchase vehicle insurance.

Stakeholders

Stakeholder	Goal
Marketing Team	Target interested customers
Sales Team	Increase policy sales
Customers	Receive relevant offers
Insurance Company	Improve revenue and reduce campaign costs

2. Formulate the requirement specifications and establish measurable goals for the proposed ML-based application using GR4ML concepts.

2.1 Functional Requirements

- FR1: System shall load customer information.
 - FR2: System shall preprocess the data.
 - FR3: System shall engineer additional features.
 - FR4: System shall predict purchase probability.
 - FR5: System shall display probability score.
 - FR6: System shall explain prediction using SHAP.
 - FR7: System shall allow batch prediction.
 - FR8: System shall save prediction history.
-

2.2 Non-Functional Requirements

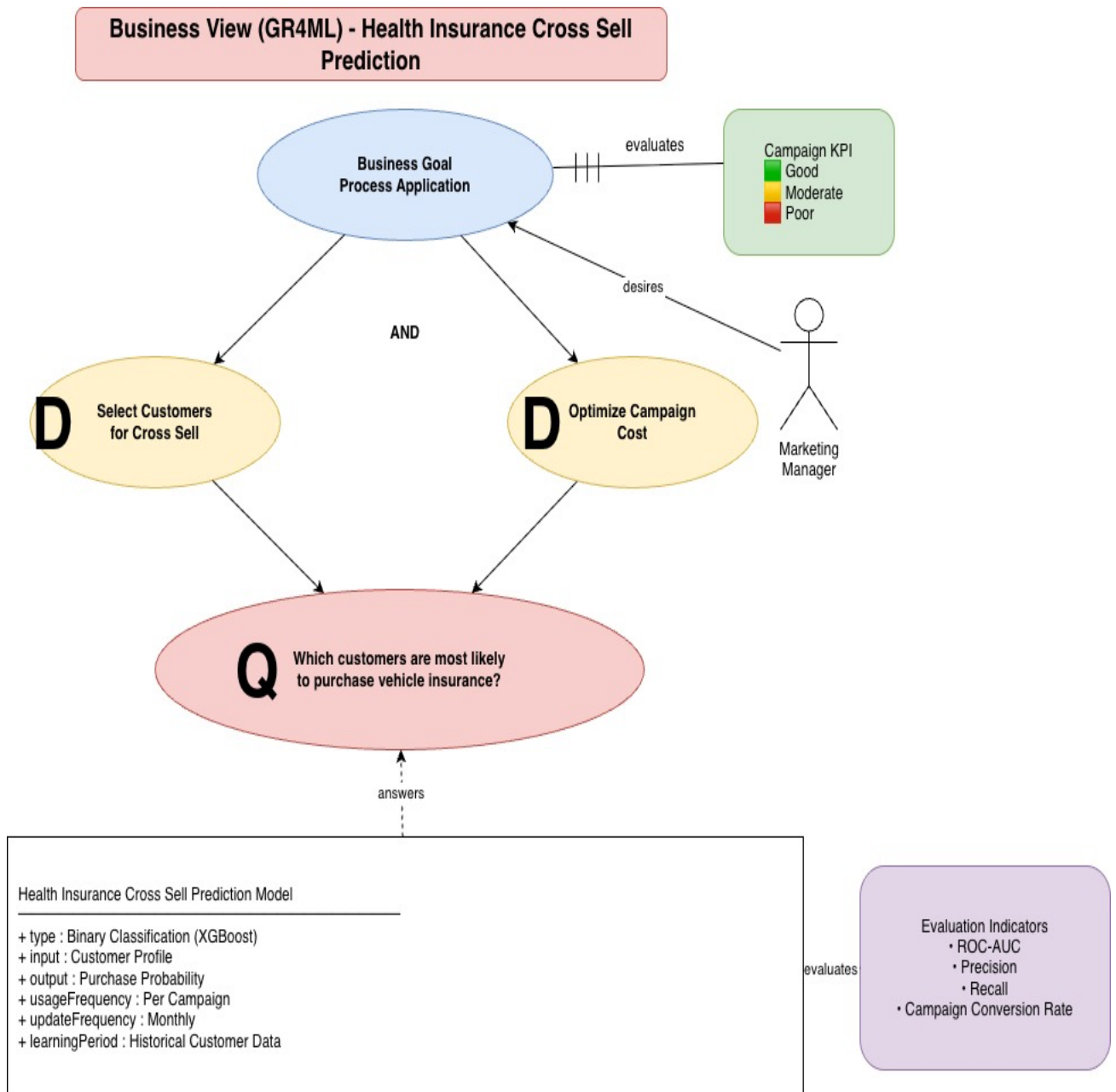
- NFR1: Prediction latency < 500 ms
 - NFR2: Availability > 99%
 - NFR3: Secure REST APIs
 - NFR4: Explainable predictions
 - NFR5: Scalable architecture
 - NFR6: Maintainability
 - NFR7: Reproducibility
-

2.3 Measurable Goals

Goal	Target
ROC-AUC	>0.88
Recall	>75%
Precision	>40%
Response Time	<500 ms
Uptime	99%
Batch Prediction	10000 customers

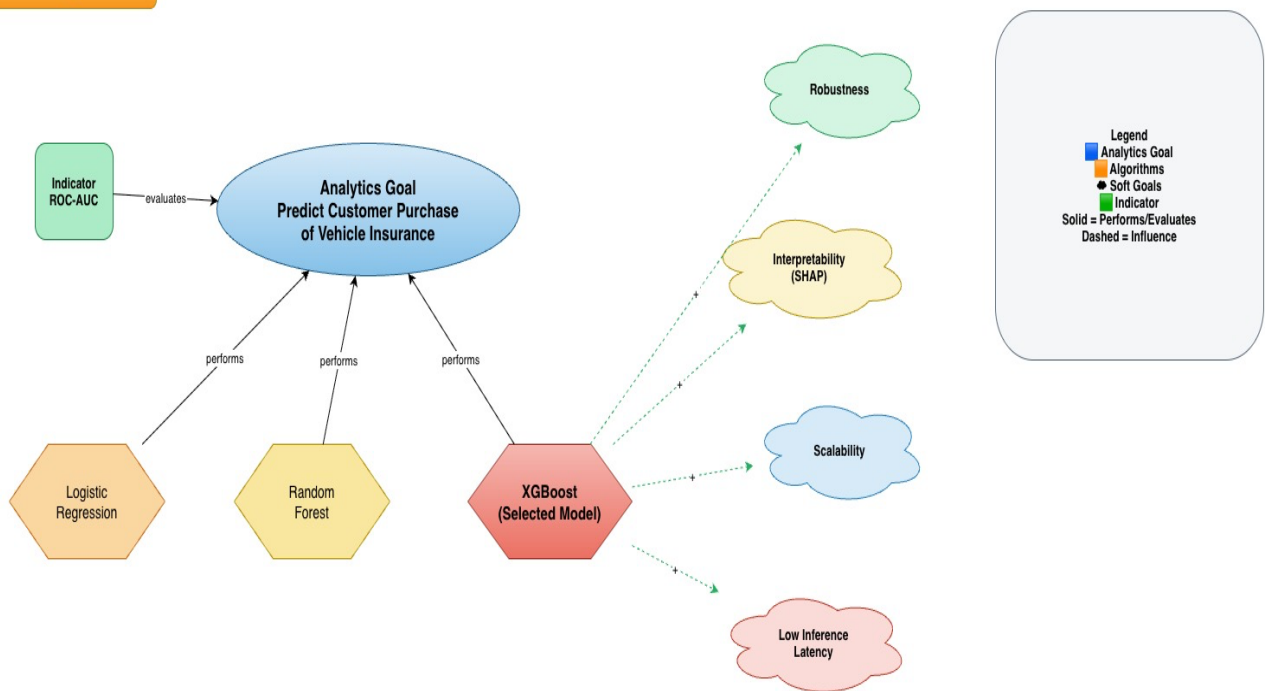
3. Using GR4ML notations, develop the following views for the application:

3.1 Business View



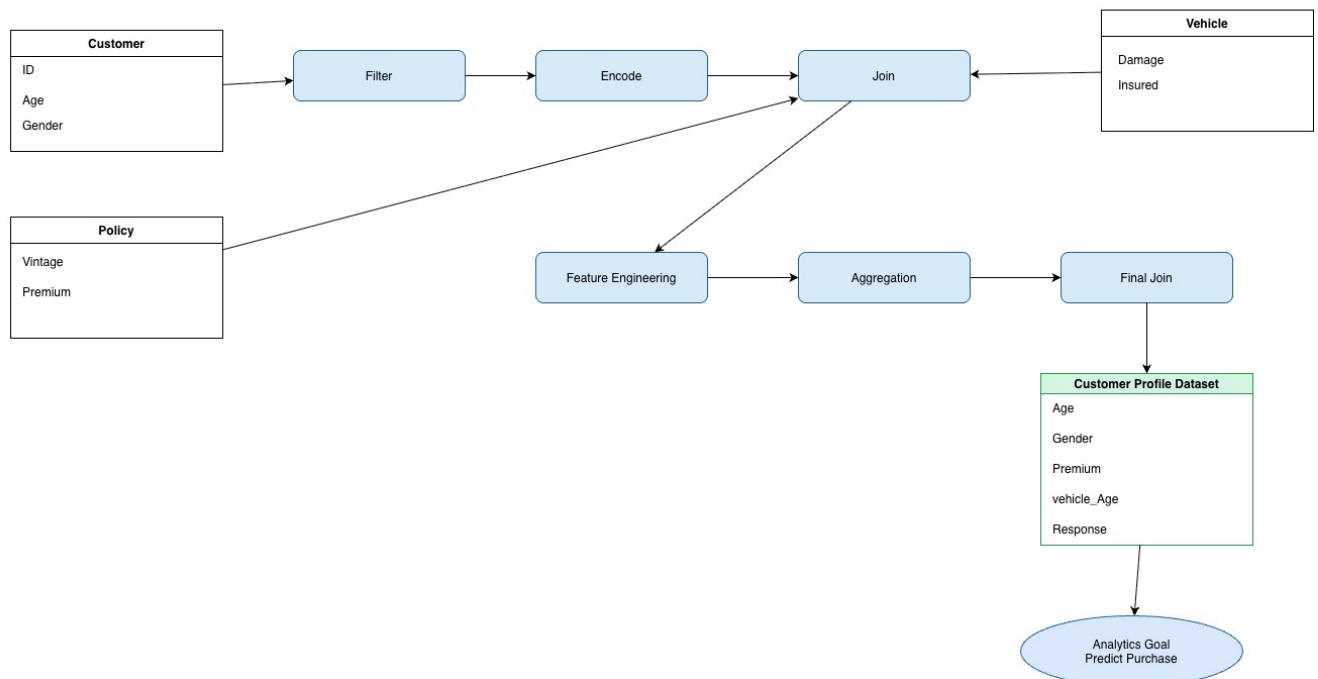
3.2 Analytics Design View

Analytics Design View



3.3 Data Preparation View

Data Preparation View



4. Identify the top three quality requirements for the application and provide justification for their selection.

4.1 Accuracy

Reason: Wrong prediction results in contacting uninterested customers, increasing operational costs.

Target: ROC-AUC > 0.88

4.2 Explainability

Reason: Insurance companies must justify recommendations and support business decisions.

Technology: SHAP

3. Scalability

Reason: The model should support predictions for millions of customers without significant performance degradation.

Technology: FastAPI + Docker + Azure

Objective 2: System Architecture

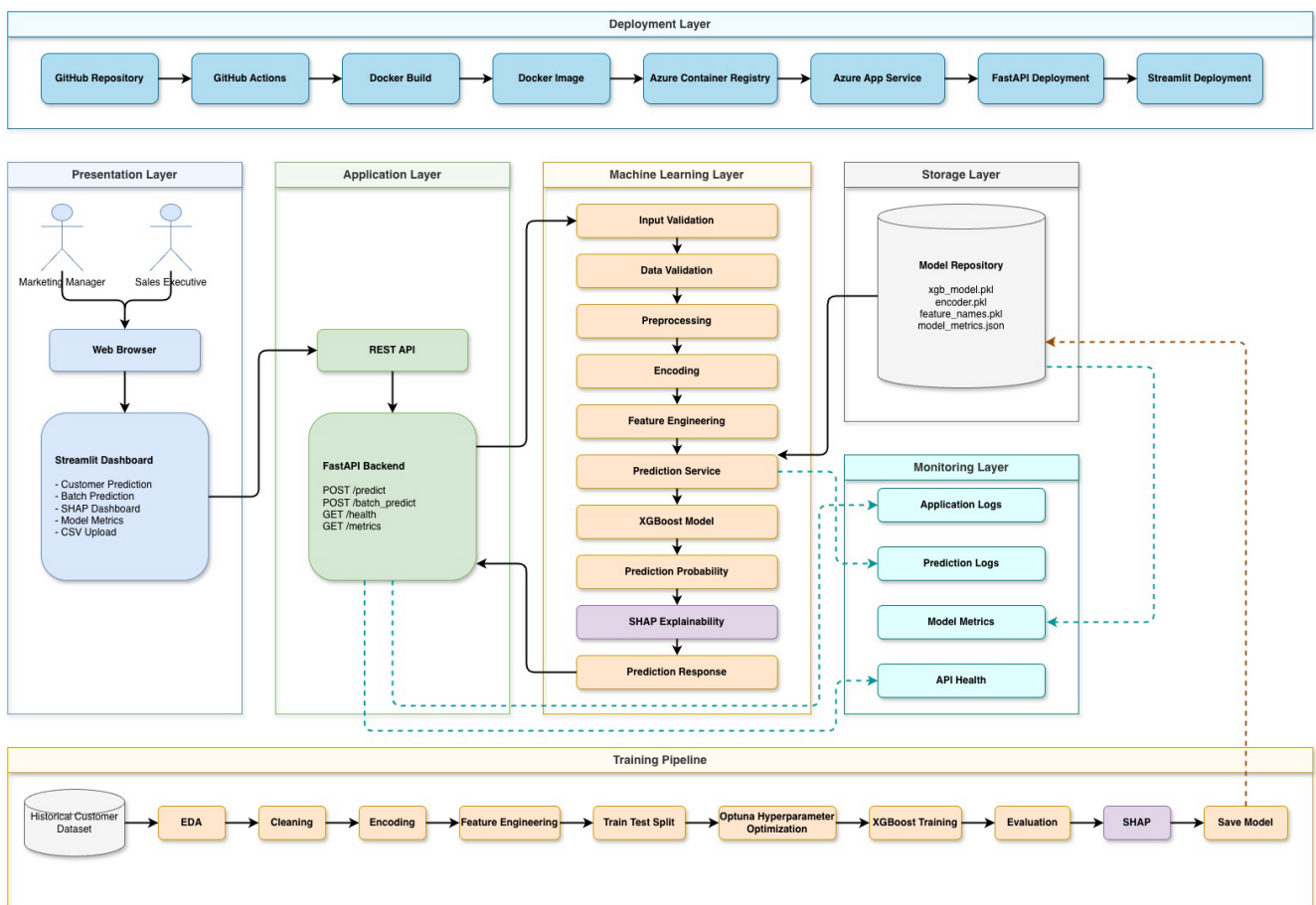
5. Draw a system architecture diagram for the selected application, clearly illustrating both ML and non-ML components. You may use any suitable tool of your choice to create the diagram.

5.1 ML Components

- Data Validation
- Feature Engineering
- XGBoost Model
- SHAP Explainability
- Prediction Service

5.2 Non-ML Components

- Streamlit UI
- FastAPI Backend
- REST APIs
- Logging w
- Model Repository
- Docker
- GitHub Actions
- Azure Deployment



6. Select and apply any two relevant architectural patterns suitable for the application.

6.1 Layered Architecture (N-Tier Architecture)

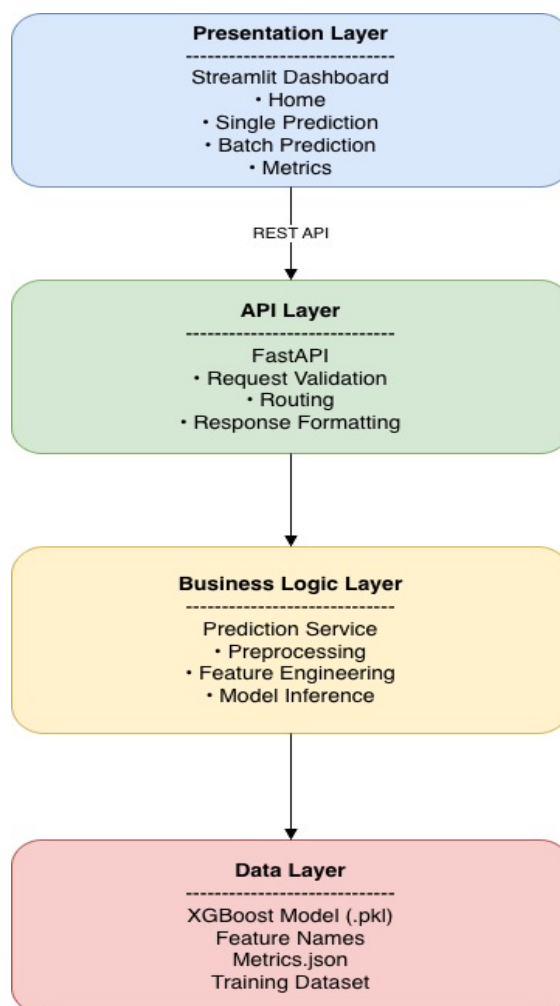
Purpose: Organizes the application into independent layers with clear responsibilities.

Application:

- Presentation Layer → Streamlit
- API Layer → FastAPI
- Business Layer → Prediction Pipeline
- Data Layer → Model files and datasets

Benefits:

- Maintainability
- Scalability
- Loose coupling
- Easier testing



6.2 Pipe and Filter Architecture

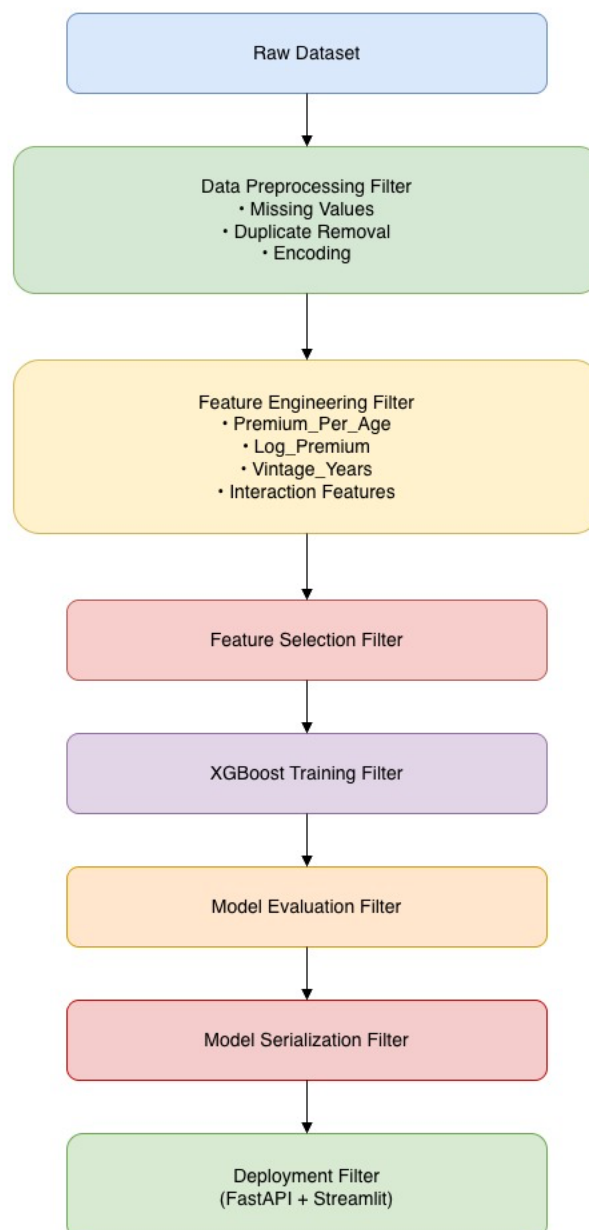
Purpose: Models the machine learning pipeline as a sequence of processing stages.

Application:

- Data Loading
- Data Preprocessing
- Feature Engineering
- Model Training
- Model Evaluation
- Deployment

Benefits:

- Modular pipeline
- Reusable processing stages
- Easy to extend
- Clear data flow



7. Implement the selected architectural patterns using appropriate technologies and tools of your choice.

7.1 Model Training

Folder structure:

```
src/
├── __init__.py          # Marks src as a Python package
├── config.py            # Project configuration and constants
├── preprocessing.py     # Data cleaning and preprocessing
├── feature_engineering.py # Feature engineering pipeline
├── predict.py          # Prediction and inference logic
└── train.py            # Model training pipeline
```

src/config.py: use for keeping all properties at one place.

src/preprocessing.py:

```
"""
preprocessing.py
Project: Health Insurance Cross Sell Prediction
This module contains all preprocessing functions.
"""

import joblib
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder

# =====
# CONFIG
# =====
from src.config import ENCODER_PATH, TARGET, TRAIN_DATA_PATH, TEST_SIZE,
RANDOM_STATE

# =====
# LOAD DATA
# =====
def load_data(path: str) -> pd.DataFrame:
    """
    Load CSV dataset
    """
    return pd.read_csv(path)

# =====
# REMOVE DUPLICATES
# =====
```

```

def remove_duplicates(df: pd.DataFrame) -> pd.DataFrame:
    before = len(df)
    df = df.drop_duplicates()
    after = len(df)
    print(f"Removed {before-after} duplicate rows")
    return df

# =====
# MISSING VALUES
# =====
def check_missing_values(df: pd.DataFrame):
    missing = pd.DataFrame({
        "Missing": df.isnull().sum(),
        "Percentage": (df.isnull().mean()*100).round(2)
    })
    return missing

# =====
# DROP ID
# =====
def drop_id(df: pd.DataFrame):
    if "id" in df.columns:
        df = df.drop(columns=["id"])
    return df

# =====
# MAP CATEGORICAL FEATURES
# =====
def map_features(df: pd.DataFrame):
    gender_map = {
        "Male": 1,
        "Female": 0
    }
    vehicle_age_map = {
        "< 1 Year": 0,
        "1-2 Year": 1,
        "> 2 Years": 2
    }
    damage_map = {
        "No": 0,
        "Yes": 1
    }
    df["Gender"] = df["Gender"].map(gender_map)
    df["Vehicle_Age"] = df["Vehicle_Age"].map(vehicle_age_map)
    df["Vehicle_Damage"] = df["Vehicle_Damage"].map(damage_map)
    return df

# =====
# CREATE AGE GROUP

```

```

# =====
def create_age_group(df):
    df["Age_Group"] = pd.cut(
        df["Age"],
        bins=[18,25,35,45,55,65,100],
        labels=[
            "18-25",
            "26-35",
            "36-45",
            "46-55",
            "56-65",
            "65+"
        ]
    )
    return df

# =====
# ORDINAL ENCODER
# =====
def fit_encoder(df):
    encoder = OrdinalEncoder()
    df[["Age_Group"]] = encoder.fit_transform(
        df[["Age_Group"]]
    )
    joblib.dump(
        encoder,
        ENCODER_PATH
    )
    return df

def transform_encoder(df):
    encoder = joblib.load(
        ENCODER_PATH
    )
    df[["Age_Group"]] = encoder.transform(
        df[["Age_Group"]]
    )
    return df

# =====
# SPLIT DATA
# =====
def split_data(df):
    X = df.drop(columns=[TARGET])
    y = df[TARGET]
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,

```

```

        test_size=TEST_SIZE,
        random_state=RANDOM_STATE,
        stratify=y
    )
    return X_train, X_test, y_train, y_test

# =====
# COMPLETE TRAINING PIPELINE
# =====
def preprocess_training_data(df):
    df = remove_duplicates(df)
    df = drop_id(df)
    df = map_features(df)
    df = create_age_group(df)
    df = fit_encoder(df)
    return split_data(df)

# =====
# COMPLETE INFERENCE PIPELINE
# =====
def preprocess_inference_data(df):
    df = drop_id(df)
    df = map_features(df)
    df = create_age_group(df)
    df = transform_encoder(df)
    return df

# =====
# MAIN
# =====
if __name__ == "__main__":
    train = load_data(TRAIN_DATA_PATH)
    print(train.head())
    print(check_missing_values(train))
    X_train, X_test, y_train, y_test = preprocess_training_data(train)
    print(X_train.shape)
    print(X_train.head())
    print(X_test.shape)

```

src/feature_engineering.py:

```

"""
feature_engineering.py Feature Engineering for Health Insurance Cross Sell
Prediction
"""
import numpy as np
import pandas as pd

# =====

```

```

# PREMIUM PER AGE
# =====
def premium_per_age(df: pd.DataFrame):

    df["Premium_Per_Age"] = (
        df["Annual_Premium"] /
        df["Age"].replace(0, 1)
    )
    return df

# =====
# CUSTOMER TENURE IN YEARS
# =====
def vintage_years(df: pd.DataFrame):
    df["Vintage_Years"] = (
        df["Vintage"] / 365
    ).round(2)
    return df

# =====
# LOG TRANSFORM PREMIUM
# =====
def log_premium(df: pd.DataFrame):
    df["Log_Premium"] = np.log1p(
        df["Annual_Premium"]
    )
    return df

# =====
# AGE FLAGS
# =====
def age_flags(df: pd.DataFrame):
    df["Young_Driver"] = (
        df["Age"] < 30
    ).astype(int)
    df["Senior_Citizen"] = (
        df["Age"] >= 60
    ).astype(int)
    return df

# =====
# HIGH PREMIUM FLAG
# Use a FIXED threshold obtained from training dataset.
# =====
HIGH_PREMIUM_THRESHOLD = 41000
def high_premium(df: pd.DataFrame):
    df["High_Premium"] = (
        df["Annual_Premium"] >= HIGH_PREMIUM_THRESHOLD
    ).astype(int)

```

```

    return df

# =====
# VEHICLE DAMAGE + NOT PREVIOUSLY INSURED
# =====
def damage_not_insured(df: pd.DataFrame):
    df["Damage_Not_Insured"] = (
        (df["Vehicle_Damage"] == 1) &
        (df["Previously_Insured"] == 0)
    ).astype(int)
    return df

# =====
# AGE × VEHICLE AGE
# =====
def age_vehicle_interaction(df: pd.DataFrame):
    df["Age_VehicleAge"] = (
        df["Age"] *
        df["Vehicle_Age"]
    )
    return df

# =====
# PREMIUM × VINTAGE
# =====
def premium_vintage(df: pd.DataFrame):
    df["Premium_Vintage"] = (
        df["Annual_Premium"] *
        df["Vintage"]
    )
    return df

# =====
# LICENSE × PREVIOUSLY INSURED
# =====
def insurance_interaction(df: pd.DataFrame):
    df["License_Insured"] = (
        df["Driving_License"] *
        df["Previously_Insured"]
    )
    return df

# =====
# AGE × PREMIUM
# =====
def age_premium(df: pd.DataFrame):
    df["Age_Premium"] = (
        df["Age"] *
        df["Annual_Premium"]
    )

```

```

    )
    return df

# =====
# COMPLETE FEATURE ENGINEERING
# =====
def feature_engineering(df: pd.DataFrame):
    df = premium_per_age(df)
    df = vintage_years(df)
    df = log_premium(df)
    df = age_flags(df)
    df = high_premium(df)
    df = damage_not_insured(df)
    df = age_vehicle_interaction(df)
    df = premium_vintage(df)
    df = insurance_interaction(df)
    df = age_premium(df)
    return df

```

src/train.py:

```

"""
train.py
Train XGBoost model for
Health Insurance Cross Sell Prediction.
"""
import json
import joblib
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)
from src.preprocessing import (
    load_data,
    preprocess_training_data
)
from src.feature_engineering import (
    feature_engineering
)
from src.config import (
    TRAIN_DATA_PATH,
    MODEL_PATH,
    FEATURE_NAMES_PATH,

```



```

METRICS_PATH,
RANDOM_STATE
)
def train():
    print("=" * 60)
    print("Loading Dataset")
    print("=" * 60)
    df = load_data(TRAIN_DATA_PATH)
    print(f"Dataset Shape : {df.shape}")
    print("=" * 60)
    print("Preprocessing")
    print("=" * 60)
    X_train, X_test, y_train, y_test = preprocess_training_data(df)
    print("=" * 60)
    print("Feature Engineering")
    print("=" * 60)
    X_train = feature_engineering(X_train)
    X_test = feature_engineering(X_test)
    # Handle Imbalanced Dataset
    negative = (y_train == 0).sum()
    positive = (y_train == 1).sum()
    scale_pos_weight = negative / positive
    print(f"Scale Pos Weight : {scale_pos_weight:.2f}")
    print("=" * 60)
    print("Training XGBoost")
    print("=" * 60)
    model = XGBClassifier(
        objective="binary:logistic",
        eval_metric="auc",
        random_state=RANDOM_STATE,
        n_estimators=400,
        learning_rate=0.03,
        max_depth=4,
        min_child_weight=7,
        subsample=0.8,
        colsample_bytree=0.8,
        gamma=2,
        reg_alpha=0.5,
        reg_lambda=8,
        scale_pos_weight=3.5,
        n_jobs=-1
    )
    model.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=100)

    print("Training Completed")
    print("=" * 60)
    print("Evaluation")
    print("=" * 60)
    #y_pred = model.predict(X_test)

```

```

y_prob = model.predict_proba(X_test)[: , 1]
y_pred = (y_prob >= 0.40).astype(int)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_prob)
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")
print(f"ROC AUC : {roc_auc:.4f}")
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report")
print(classification_report(y_test, y_pred))
print("=" * 60)
print("Saving Model")
print("=" * 60)
joblib.dump(model, MODEL_PATH)
joblib.dump(list(X_train.columns), FEATURE_NAMES_PATH)
metrics = {
    "model": "XGBoost",
    "dataset": "Health Insurance Cross Sell",
    "train_samples": int(len(X_train)),
    "test_samples": int(len(X_test)),
    "features": list(X_train.columns),
    "accuracy": float(accuracy),
    "precision": float(precision),
    "recall": float(recall),
    "f1_score": float(f1),
    "roc_auc": float(roc_auc),
    "confusion_matrix": confusion_matrix(
        y_test,
        y_pred
    ).tolist()
}
with open(METRICS_PATH, "w") as f:
    json.dump(metrics, f)
print(f"Model Saved : {MODEL_PATH}")
print(f"Feature Names Saved : {FEATURE_NAMES_PATH}")
print(f"Metrics Saved : {METRICS_PATH}")
print("=" * 60)
print("Training Finished")
print("=" * 60)

if __name__ == "__main__":
    train()

```

Training completion screenshot:

```
(.venv) (base) rishabhjain@Rishabhs-MacBook-Pro Health-Insurance-Cross-Sell-Prediction % python -m src.train
=====
Preprocessing
=====
Removed 0 duplicate rows
=====
Feature Engineering
=====
Scale Pos Weight : 7.16
=====
Training XGBoost
=====
[0] validation_0-auc:0.84023
[100] validation_0-auc:0.85358
[200] validation_0-auc:0.85687
[300] validation_0-auc:0.85860
[399] validation_0-auc:0.85934
Training Completed
=====
Evaluation
=====
Accuracy : 0.7346
Precision : 0.3012
Recall : 0.8827
F1 Score : 0.4492
ROC AUC : 0.8593

Confusion Matrix
[[23875 9565]
 [ 548 4123]]

Classification Report
precision recall f1-score support
 0 0.98 0.71 0.83 33440
 1 0.30 0.88 0.45 4671

 accuracy 0.73 38111
 macro avg 0.64 0.80 0.64 38111
 weighted avg 0.89 0.73 0.78 38111

=====
Saving Model
=====
Model Saved : /Users/rishabhjain/Documents/Bits-Pilani/Sem2/SEML/Health-Insurance-Cross-Sell-Prediction/models/xgb_model.pkl
Feature Names Saved : /Users/rishabhjain/Documents/Bits-Pilani/Sem2/SEML/Health-Insurance-Cross-Sell-Prediction/models/feature_names.pkl
Metrics Saved : /Users/rishabhjain/Documents/Bits-Pilani/Sem2/SEML/Health-Insurance-Cross-Sell-Prediction/models/metrics.json
=====
Training Finished
=====
```

src/predict.py:

```
"""
predict.py
Prediction pipeline for
Health Insurance Cross Sell Prediction
"""

import joblib
import pandas as pd
from src.config import (
    MODEL_PATH,
    FEATURE_NAMES_PATH
)
from src.preprocessing import (
    preprocess_inference_data
)
from src.feature_engineering import (
    feature_engineering
)

class Predictor:
    def __init__(self):
        self.model = joblib.load(MODEL_PATH)
```

```

        self.feature_names = joblib.load(FEATURE_NAMES_PATH)
# =====
# Prepare Data
# =====
def prepare(self, df: pd.DataFrame):
    df = preprocess_inference_data(df)
    df = feature_engineering(df)
    # Ensure correct feature order
    df = df[self.feature_names]
    return df

# =====
# Single Prediction
# =====
def predict_single(self, data: dict):
    df = pd.DataFrame([data])
    df = self.prepare(df)
    prediction = self.model.predict(df)[0]
    probability = self.model.predict_proba(df)[0][1]
    return {
        "prediction": int(prediction),
        "probability": round(float(probability), 4),
        "label": "Interested" if prediction == 1 else "Not Interested"
    }

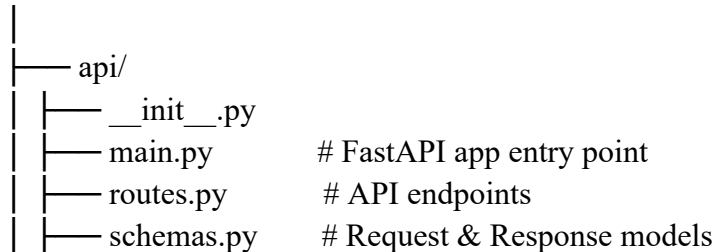
# =====
# Batch Prediction
# =====
def predict_batch(self, df: pd.DataFrame):
    df_processed = self.prepare(df)
    predictions = self.model.predict(df_processed)
    probabilities = self.model.predict_proba(df_processed)[: , 1]
    result = df.copy()
    result["Prediction"] = predictions
    result["Probability"] = probabilities
    result["Label"] = result["Prediction"].map({
        0: "Not Interested",
        1: "Interested"
    })
    return result

```

7.2 Fast API

Folder structure:

Health-Insurance-Cross-Sell-Prediction/



API URL: <https://health-insurance-cross-sell-prediction.onrender.com/docs>

api/main.py:

```
from fastapi import FastAPI
from .routes import router

app = FastAPI(
    title="Health Insurance Prediction API",
    description="This API predicts whether a customer will buy health insurance or not based on their demographic and personal information.",
    version="1.0.0",
)

app.include_router(router)
```

api/schemas.py:

```
from pydantic import BaseModel, Field

class PredictionRequest(BaseModel):
    id: int = Field(..., example=1)
    Gender: str = Field(..., example="Male")
    Age: int = Field(..., ge=18, le=100, example=35)
    Driving_License: int = Field(..., example=1)
    Region_Code: float = Field(..., example=28.0)
    Previously_Insured: int = Field(..., example=0)
    Vehicle_Age: str = Field(..., example="1-2 Year")
    Vehicle_Damage: str = Field(..., example="Yes")
    Annual_Premium: float = Field(..., example=40454.0)
    Policy_Sales_Channel: float = Field(..., example=26.0)
    Vintage: int = Field(..., example=217)

class PredictionResponse(BaseModel):
    prediction: int
    probability: float
```

```
label: str
```

api/routes.py:

```
import json
import pandas as pd
from fastapi import APIRouter, UploadFile, File
from api.schemas import (
    PredictionRequest,
    PredictionResponse
)
from src.predict import Predictor

predictor = Predictor()
router = APIRouter()

@router.get("/")
def home():
    return {"Project": "Health Insurance Prediction API running. Use /docs for API documentation.", "status": "Running"}

@router.get("/ping")
def ping():
    return {"Project": "Health Insurance Prediction API working", "status": "success", "code": 200, "version": "1.0.0"}

@router.get("/metrics")
def metrics():
    with open("models/metrics.json") as f:
        return json.load(f)

@router.post("/predict", response_model=PredictionResponse)
def predict(PredictionRequest: PredictionRequest):
    result = predictor.predict_single(
        PredictionRequest.model_dump()
    )
    return result

@router.post("/batch-predict")
async def batch_predict(
    file: UploadFile = File(...)
):
    df = pd.read_csv(file.file)
    result = predictor.predict_batch(df)
    return result.to_dict(
        orient="records")
```

API Swagger docs:

Health Insurance Prediction API 1.0.0 OAS 3.1

/openapi.json

This API predicts whether a customer will buy health insurance or not based on their demographic and personal information.

default

GET / Home

GET /ping Ping

GET /metrics Metrics

POST /predict Predict

POST /batch-predict Batch Predict

Schemas

Body_batch_predict_batch_predict_post > Expand all object

HTTPValidationError > Expand all object

PredictionRequest > Expand all object

PredictionResponse > Expand all object

7.3 Streamlit (Front-end):

Folder structure:

```
streamlit/
├── Home.py           # Landing page
├── config.py         # Streamlit configuration
└── pages/
    ├── Prediction.py # Single/Batch prediction page
    └── metrics.py    # Model evaluation dashboard
```

Home.py:

```
import streamlit as st

st.set_page_config(
    page_title="Health Insurance Cross Sell",
    page_icon="🚗",
    layout="wide"
)

st.title("🚗 Health Insurance Cross Sell Prediction")
st.markdown("----")
st.write(
    """
    This application predicts whether a customer is likely to purchase
    Vehicle Insurance using an XGBoost model.
    ### Features
    - Single Prediction
    - Batch Prediction
    - Model Metrics
    - FastAPI Backend
    - XGBoost
    """
)
```

config.py:

```
from pathlib import Path

# =====
# PROJECT ROOT
# =====
PROJECT_ROOT = Path(__file__).resolve().parent.parent
print(f"Project Root: {PROJECT_ROOT}")

# =====
# DATA PATHS
# =====
```



```

DATA_DIR = PROJECT_ROOT / "dataset"
TEST_DATA_PATH = DATA_DIR / "test.csv"

# =====
# API ENDPOINT SETTINGS
# =====
BASE_URL = "https://health-insurance-cross-sell-prediction.onrender.com"
METRICS_URL = BASE_URL + "/metrics"
SINGLE_PREDICTION_URL = BASE_URL + "/predict"
BATCH_PREDICTION_URL = BASE_URL + "/batch-predict"

```

prediction.py:

```

import streamlit as st
import requests
from pathlib import Path
from config import SINGLE_PREDICTION_URL, BATCH_PREDICTION_URL, TEST_DATA_PATH

predictionMethod = st.selectbox("Prediction Method", ["--Select--", "Single", "Batch"])

if predictionMethod == "Single":
    st.write("You selected Single Prediction")
    age = st.number_input("Age", min_value=0, max_value=120, value=30)
    gender = st.selectbox("Gender", ["Male", "Female"])
    id = st.number_input("id", min_value=0, max_value=120, value=30)
    Driving_License = st.checkbox("Driving License")
    Region_Code = st.number_input("Region Code", value=28.0)
    Previously_Insured = st.number_input("Previously Insured", min_value=0, max_value=1, value=0)
    Vehicle_Age = st.selectbox("Vehicle Age", ["< 1 Year", "1-2 Year", "> 2 Years"])
    Vehicle_Damage = st.selectbox("Vehicle Damage", ["No", "Yes"])
    Annual_Premium = st.number_input("Annual Premium", value=40454.0)
    Policy_Sales_Channel = st.number_input("Policy Sales Channel", value=26.0)
    Vintage = st.number_input("Vintage", min_value=0, max_value=1000, value=217)

    if st.button("Predict"):
        payload = {
            "Age": age,
            "Gender": gender,
            "id": id,
            "Driving_License": Driving_License,
            "Region_Code": Region_Code,
            "Previously_Insured": Previously_Insured,
            "Vehicle_Age": Vehicle_Age,
            "Vehicle_Damage": Vehicle_Damage,
            "Annual_Premium": Annual_Premium,

```

```

        "Policy_Sales_Channel": Policy_Sales_Channel,
        "Vintage": Vintage
    }

    response = requests.post(SINGLE_PREDICTION_URL, json=payload)
    result = response.json()
    print(result)
    st.success(result["label"])
    st.metric(
        "Probability",
        f"{result['probability']:.2%}"
    )

if predictionMethod == "Batch":
    st.write("You selected Batch Prediction")
    with open(TEST_DATA_PATH, "rb") as file:
        st.download_button(
            label="📄 Download Sample CSV",
            data=file,
            file_name="sample_input.csv",
            mime="text/csv",
            use_container_width=True
        )

    uploaded = st.file_uploader(
        "Upload CSV",
        type="csv"
    )

    if uploaded:
        print("Uploaded file:", uploaded.name)
        files = {
            "file": (uploaded.name, uploaded.getvalue(), "text/csv")
        }
        headers = {
            "accept": "application/json"
        }
        response = requests.post(BATCH_PREDICTION_URL, headers=headers,
files=files)
        print("Status Code:", response.status_code)
        try:
            result = response.json()
        except Exception:
            print(response.text)
        st.dataframe(result)

```

Prediction solution screenshots:

#screenshot 1: Initial page

Prediction Method

--Select--



--Select--

Single

Batch

#screenshot 2: After selecting Batch as option and result

Prediction Method

Batch



You selected Batch Prediction



Download Sample CSV

Upload CSV



test.csv
27.4KB



Vehicle_Damage	Annual_Premium	Policy_Sales_Channel	Vintage	Prediction	Probability	Label
No	35786	152	53	0	0.0026	Not Interested
Yes	33762	7	111	1	0.6238	Interested
Yes	40050	124	199	1	0.6058	Interested
Yes	37356	152	187	0	0.0212	Not Interested
No	59097	152	297	0	0.0027	Not Interested
No	40207	152	266	0	0.0018	Not Interested
No	40118	26	107	0	0.0019	Not Interested
No	23375	152	232	0	0.0019	Not Interested
Yes	2630	26	277	1	0.6319	Interested
No	27124	124	133	0	0.0036	Not Interested
...						

#screenshot 2: After selecting Single as option and result

Single



You selected Single Prediction

Age

30



Gender

Male



id

30



☐ Driving License

Region Code

28.00



Previously Insured

0



Vehicle Age

< 1 Year



Vehicle Damage

Yes



Annual Premium

40454.00



Policy Sales Channel

26.00



Vintage

217



Predict

Interested

Probability

59.41%

Metrics.py:

```
import requests
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
from config import METRICS_URL

# -----
# Configuration
# -----

st.set_page_config(
    page_title="Model Metrics",
    page_icon="📊",
    layout="wide"
)
st.title("📊 Model Performance Dashboard")
st.markdown("----")

# -----
# Get Metrics
# -----

try:
    response = requests.get(METRICS_URL)
    if response.status_code != 200:
        st.error("Unable to fetch metrics from FastAPI.")
        st.stop()
    metrics = response.json()
except Exception as e:
    st.error(f"Connection Error: {e}")
    st.stop()

# -----
# Model Information
# -----

st.subheader("📌 Model Information")
col1, col2 = st.columns(2)
with col1:
    st.info(f"**Model:** {metrics['model']}")
    st.info(f"**Dataset:** {metrics['dataset']}")
with col2:
    st.info(f"**Training Samples:** {metrics['train_samples']:,}")
    st.info(f"**Testing Samples:** {metrics['test_samples']:,}")
st.markdown("----")

# -----
```

```

# Performance Metrics
# -----
st.subheader("📊 Performance Metrics")
c1, c2, c3 = st.columns(3)
c1.metric( "Accuracy", f"{metrics['accuracy']:.4f}")
c2.metric( "Precision", f"{metrics['precision']:.4f}")
c3.metric( "Recall", f"{metrics['recall']:.4f}")

c4, c5, c6 = st.columns(3)
c4.metric( "F1 Score", f"{metrics['f1_score']:.4f}")
c5.metric( "ROC AUC", f"{metrics['roc_auc']:.4f}")
st.markdown("----")

# -----
# Confusion Matrix
# -----
st.subheader("🇮🇹 Confusion Matrix")
cm = np.array(metrics["confusion_matrix"])
fig, ax = plt.subplots(figsize=(5, 5))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not Interested", "Interested"]
)
disp.plot(ax=ax, cmap="Blues", colorbar=False, values_format="d")
ax.set_title("Confusion Matrix", fontsize=12)
plt.tight_layout(pad=0.5)
st.pyplot(fig, use_container_width=False)
st.markdown("----")

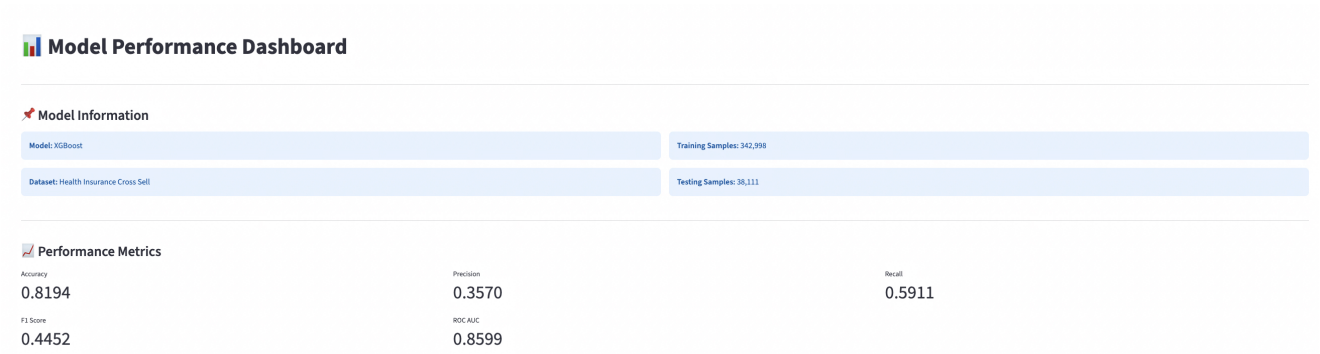
# -----
# Feature List
# -----
st.subheader("🧠 Features Used for Training")
feature_df = pd.DataFrame(
    {
        "Feature Name": metrics["features"]
    }
)
st.dataframe(
    feature_df,
    use_container_width=True,
    hide_index=True
)
st.markdown("----")

# -----
# Summary
# -----
st.subheader("📋 Summary")

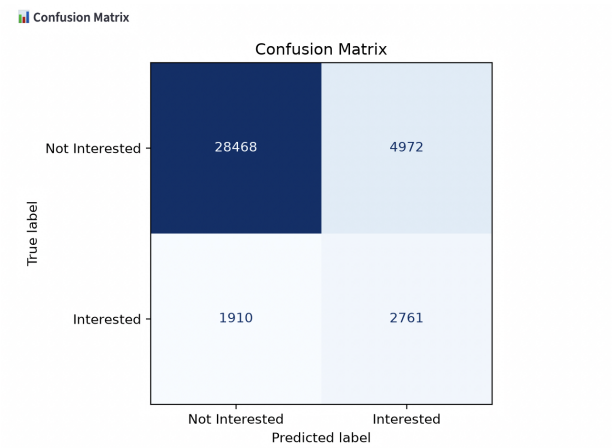
```

```
summary = pd.DataFrame(
    {
        "Metric": [
            "Accuracy",
            "Precision",
            "Recall",
            "F1 Score",
            "ROC AUC"
        ],
        "Value": [
            metrics["accuracy"],
            metrics["precision"],
            metrics["recall"],
            metrics["f1_score"],
            metrics["roc_auc"]
        ]
    }
)
st.dataframe(
    summary,
    use_container_width=True,
    hide_index=True
)
```

#screenshot 1:



#screenshot 2:



Future scope:

Future Enhancement	Benefit
Advanced ML Models	Improve prediction accuracy and robustness
Real-Time Prediction	Instant customer scoring
Cloud Deployment	Scalability and high availability
MLOps Pipeline	Automated retraining and deployment
Explainable AI	Better model transparency
Customer Segmentation	Targeted marketing campaigns
Recommendation System	Personalized insurance offers
Fraud Detection	Reduced fraudulent claims
Model Monitoring	Detect model and data drift
Multi-Platform Integration	Easy integration with enterprise systems
Enhanced Dashboard	Better business insights
Security & Authentication	Enterprise-grade application security